

基础三大查找

基本概念

静态查找 (Static Search)

数据集合稳定，不需要添加，删除元素的查找操作。

动态查找 (Dynamic Search)

在数据集合中进行查找的过程中，需要同时添加或删除元素的查找操作。

关键字 (key)

数据元素（或记录）中某个数据项的值，用它可以唯一的标识（识别）一个数据元素（或记录）。

内查找和外查找

查找也有内查找和外查找之分。若整个查找过程都在内存中进行，则称之为**内查找**(internal search)；反之，若查找过程的需要访问外存，则称之为**外查找**(external search)。

平均查找长度

在查找运算中时间主要花费在关键字的比较上，把平均需要和给定值进行比较的关键字次数称为平均查找长度(Average Search Length, ASL)，其定义如下：

$$ASL = \sum_{i=1}^n p_i c_i$$

其中，n是查找表中元素的个数。 p_i 是查找第*i*个元素的概率，通常假设每个元素的查找概率相等，此时 $p = 1/n$ (1 ≤ i ≤ n)， c_i 是找到第*i*个元素所需的关键字比较次数。

ASL分为查找成功情况下ASL和查找不成功(失败)情况下的ASL。

ASL（成功）表示成功查找到查找表中的元素，平均需要关键字比较次数（ p_i 为查找到第*i*个元素的概率）

$$\sum_{i=1}^n p_i = 1$$

ASL（失败）表示没有查找到查找表中的元素，平均需要关键字的比较次数（假设有m种查找失败的情况， q_i 为第*i*种情况的概率）

$$\sum_{i=1}^m q_i = 1$$

显然，ASL是衡量查找算法性能好坏的重要指标。一个查找算法的ASL越大，其时间性能越差；反之，一个查找算法的ASL越小，其时间性能越好。

查找算法

对于静态查找来说，我们可以采用用线性表结构组织数据，这样可以使顺序查找算法，如果我们对关键字进行排序，则可以使用折半查找算法或斐波那契查找算法等来提高查找的效率。

对于动态查找来说，我们则可以考 虑使用二叉排序树的查找技术，另外我们还可以使用散列表结构来解决一些查找问题。

查找的分类

其实本质上查找算法一般分为两类：

顺序查找（Sequential Search）：将顺序遍历链表或数组，并检查每个元素。（比如线性查找）

区间查找（Interval Search）：这类算法是专门为查找排序的数据结构而设计的。这些类型的查找算法比线性查找有效得多，因为它们反复检查查找空间的中心并不断将查找空间缩减。（比如二分查找）

顺序查找

基本思想

顺序查找（Sequential Search）又叫线性查找（Linear Search），是最基本的查找技术，它的查找过程为：从表中的第一个（或者最后一个）记录开始，逐个进行记录的关键字和给定值的比较，若某个记录的关键字和给定值比较相等，则查找成功。如果查找了所有的记录仍然找不到与给定值相等的关键字，则查找不成功。

顺序查找的优化

被查概率不相等时，将被查概率大的放在靠前的位置。该方法能在查找成功的情况下，平均查找长度进一步的缩短，但是对于查找失败的情况，平均查找长度没有改变。

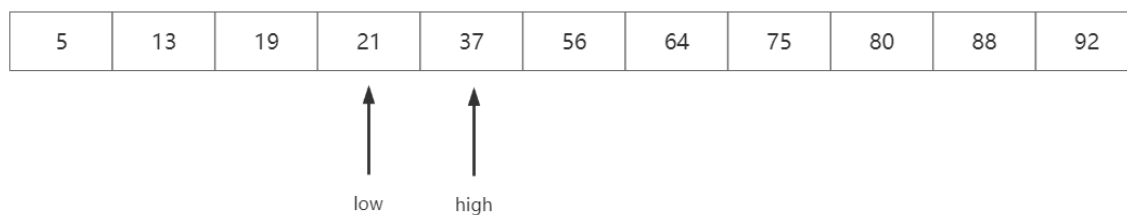
折半查找（二分查找）

基本思想

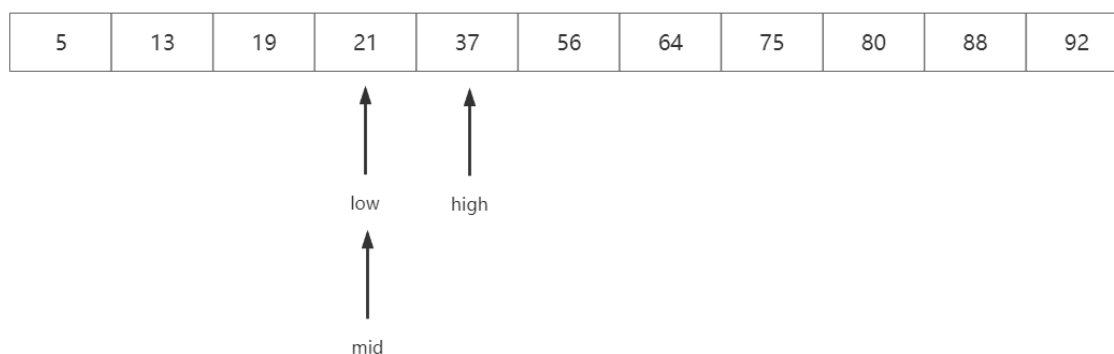
二分查找又叫做折半查找，其查找过程为：先确定待查记录所在的范围（区间），然后逐步缩小范围知道找到或者找不到该记录为止。注意二分查找是在有序表上进行的，且二分查找也是分治思想的很好例证。

5	13	19	21	37	56	64	75	80	88	92
---	----	----	----	----	----	----	----	----	----	----

我们以上面的有序数组，查找关键字21的数据元素为例进行说明。



第六步：重新计算mid 的值， $\text{mid} = \lfloor (3+4)/2 \rfloor = 3$ 。



第七步：将给定值21与mid指向的值21进行比较，发现相等，查找成功。

从上面的例子可见，折半查找过程以处于区间中间位置mid记录的值和给定值比较，若相等，则查找成功，若不相等，则缩小范围，直至新的区间中间位置mid记录的值等于给定值或者查找区间的大小小于零时（表示查找不成功）为止。

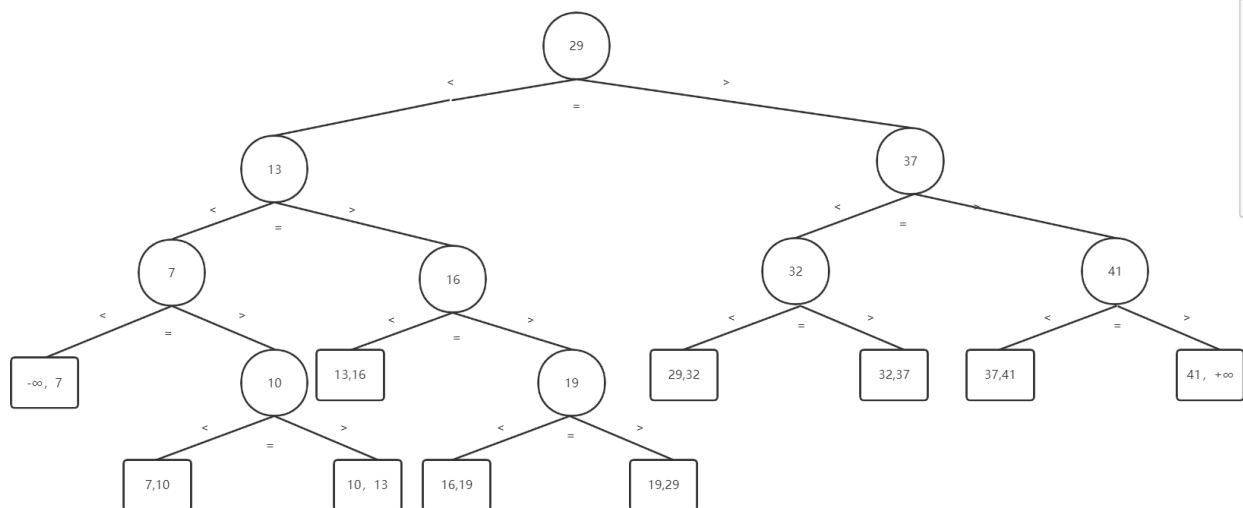
程序分析

因为折半查找需要方便地定位查找区域，所以它要求线性表必须具有随机存取的特性。因此，该查找法仅适合于顺序存储结构，不适合于链式存储结构，且要求元素按关键字有序排列。注意：不能说折半查找不能用于链式存储结构，只是说采用顺序表时折半查找算法设计更方便，效率更高。

判定树

折半查找过程可用二叉树来描述，把当前查找区间的中间位置上的元素作为根，由左子表和右子表构造的二叉树分别作为根的左子树和右子树，由此得到的二叉树称为描述折半查找过程的判定树 (decision tree) 或比较树 (comparison tree)。

树中每个圆形结点表示一个记录，结点中的值为该记录的关键字值；树中最下面的叶结点都是方形的，它表示查找不成功的情况。从判定树可以看出，查找成功时的查找长度为从根结点到目的结点的路径上的结点数，而查找不成功时的查找长度为从根结点到对应失败结点的父结点的路径上的结点数；每个结点值均大于其左子结点值，且均小于其右子结点值。若有序序列有n个元素，则对应的判定树有n个圆形的非叶结点和n+1个方形的叶结点。显然，判定树是一棵平衡二叉树。



索引存储结构和分块查找

索引存储结构

索引存储结构是在存储数据的同时还建立附加的索引表。索引表中的每一项称为索引项，索引项的一般形式为(关键字，地址)。其中，关键字唯一标识一个结点，地址作为指向该关键字对应结点的指针，也可以是相对地址(如数组的下标)。

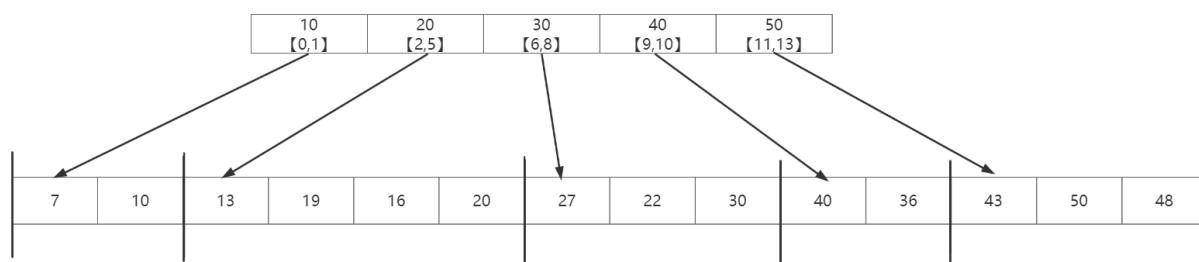
在索引存储结构中进行关键字查找时先在索引表中快速查找(因为索引表中按关键字有序排列，可以采用折半查找)到相应的关键字，然后通过对应的地址找到主数据表中的元素。索引存储结构优点是可以提高按关键字查找元素的效率；缺点是需要建立索引表而增加时间和空间的开销。

分块查找

分块查找又称索引顺序查找，它吸取了顺序查找和折半查找各自的优点，既有动态结构，又适于快速查找。

基本思想

将查找表分为若干子块。块内的元素可以无序，但块之间是有序的，即第一个块中的最大关键字小于第二个块中的所有记录的关键字，第二个块中的最大关键字小于第三个块中的所有记录的关键字，以此类推。再建立一个索引表，索引表中的每个元素含有各块的最大关键字和各块中的第一个元素的地址，索引表按关键字有序排列。



分块查找的过程分为两步：第一步是在索引表中确定待查记录所在的块，可以顺序查找或折半查找索引表；第二步是在块内顺序查找。

查找目标: 22

